AWS OPERATIONS RUNBOOK / STORAGE

Format and Mount an EBS Volume on EC2

Operational runbook for preparing a newly attached 20 GB Amazon EBS volume on a Linux EC2 instance

Scope	New, empty, single-disk EBS volume	**Filesystem**	ext4
Mount point	/data	**Persistence**	/etc/fstab

1. Objective

Prepare an attached EBS block device for Linux use by identifying the correct disk, creating an ext4 filesystem, mounting it at /data, and configuring automatic mounting after reboot using /etc/fstab.

2. Preconditions and safety

The 20 GB EBS volume is attached to the intended EC2 instance and is in the AWS console state 'in-use'.

You have SSH access and sudo privileges.

Confirm the target device carefully before running any formatting command.

Formatting an existing filesystem can destroy data. Do not format a disk that contains data you need.

3. Identify the device

List block devices:

```
lsblk
```

Inspect filesystem signatures and mount points:

```
sudo lsblk -f
sudo file -s /dev/nvme1n1
```

Example only: on many Nitro-based EC2 instances, the new disk may appear as /dev/nvme1n1. Your device name may differ. Verify the size (20 GB), filesystem, and mount point. Never assume the example device name is correct.

4. Create an ext4 filesystem

Only if you have verified that the target is the new, empty volume, create the filesystem:

```
sudo mkfs.ext4 /dev/nvme1n1
```

Verify the filesystem and record its UUID:

```
sudo blkid /dev/nvme1n1
```

Expected output includes TYPE="ext4" and a UUID. Use the actual UUID in later steps.

5. Create a mount point and mount the volume

```
sudo mkdir -p /data
```

```
sudo mount /dev/nvme1n1 /data
```

```
df -hT /data

findmnt /data
```

The output should show the new filesystem mounted at /data with approximately 20 GB capacity (reported usable space will be slightly lower due to filesystem overhead and unit conventions).

Optional write test:

sudo touch /data/test-file.txt
ls -l /data

6. Make the mount persistent across reboot

Back up the existing filesystem table before editing it:

```
sudo cp /etc/fstab /etc/fstab.bak
```

Read the actual UUID:

```
sudo blkid /dev/nvme1n1
```

Edit /etc/fstab:

```
sudo nano /etc/fstab
```

Add one line, replacing the sample UUID with the actual UUID:

```
UUID=<actual-filesystem-UUID> /data ext4 defaults,nofail 0 2
```

Field meanings:

Validate the configuration before rebooting:

```
sudo mount -a

findmnt /data

df -hT /data
```

Do not reboot if mount -a reports an error. Correct the /etc/fstab entry first. If /data is already mounted, mount -a should leave the existing mount in place.

7. Final verification checklist

■ lsblk -f shows the expected 20 GB device and ext4 filesystem.

■ findmnt /data shows the expected source mounted at /data.

■ df -hT /data reports the expected filesystem and capacity.

■ A test file can be created under /data, if write access is intended.

■ sudo mount -a completes without errors.

■ The /etc/fstab entry uses the correct UUID and has no duplicate or conflicting mount entry.

8. Troubleshooting notes

The disk is not visible in lsblk: Check the EC2 attachment and instance/volume Availability Zone in AWS. Re-run lsblk. On Nitro instances, EBS volumes commonly use NVMe device names.

mount reports an unknown filesystem: Check sudo lsblk -f and sudo blkid. Confirm the filesystem was created on the intended device.

/data shows the root filesystem: The EBS volume may not be mounted, or the mount command may have failed. Check findmnt /data and lsblk -f.

mount -a reports an error: Check the UUID, filesystem type, mount-point directory, spelling, and /etc/fstab syntax. Do not reboot until corrected.

Permission denied when writing to /data: A newly created ext4 filesystem is normally owned by root at its top level. Set ownership or permissions deliberately for the application/service account; avoid broad chmod 777.

9. Production considerations

EBS is block storage; creating a filesystem and mounting it are Linux-level operations.

EBS data persists independently of the EC2 root filesystem, subject to volume lifecycle settings and deletion configuration.

A persistent mount is not a backup. Configure and test EBS snapshots or an appropriate backup policy.

Use filesystem UUIDs in /etc/fstab rather than relying on device names that may change.

For application workloads, grant access to /data using least-privilege ownership and permissions.

The nofail option prioritizes boot continuity; if a service strictly depends on /data, configure service dependencies and health checks so it does not silently start without the volume.

10. Command summary

```
# Identify the device first; replace /dev/nvme1n1 if needed
lsblk
sudo lsblk -f
sudo file -s /dev/nvme1n1

# ONLY for a verified new/empty volume
sudo mkfs.ext4 /dev/nvme1n1

# Mount
sudo mkdir -p /data
sudo mount /dev/nvme1n1 /data
df -hT /data
findmnt /data

# Get UUID and configure persistent mounting
sudo blkid /dev/nvme1n1
sudo cp /etc/fstab /etc/fstab.bak
sudo nano /etc/fstab
# Add: UUID=<actual-filesystem-UUID> /data ext4 defaults,nofail 0 2

# Validate before reboot
sudo mount -a
findmnt /data
df -hT /data
```

Runbook scope: New, empty, single-disk EBS volume formatted as ext4 on a Linux EC2 instance. Commands and device names must be verified for the target host.

Field	Meaning
UUID=...	Stable filesystem identifier; avoids depending on device enumeration.
/data	Directory where the filesystem will be mounted.
ext4	Filesystem type.
defaults	Standard mount options.

Field	Meaning
nofail	Allows boot to continue if this volume is unavailable; applications may still fail if they require /data.
0	Disables the legacy dump backup flag.
2	Filesystem check order after the root filesystem.